

**Elemi adatszerkezetek, bináris keresőfák, hasító táblázatok,
gráfok és fák számítógépes reprezentációja**

ALAPFOGALMAK

- az algoritmusok hatékonyságát aszimptotikus jelölésekkel mérjük:
- **Ordó ($O(g(n))$) - Aszimptotikus felső korlát**
 - általában a legrosszabb esetet jelenti
 - azt jelenti, hogy az algoritmus futási ideje legfeljebb milyen gyorsan nő a bemenet (n) függvényében
 - formálisan, létezik c és n_0 pozitív konstans úgy, hogy megfelelően nagy n esetén ($n \geq n_0$), akkor $0 \leq f(n) \leq c \cdot g(n)$
 - $f(n)$ felülről korlátozza a $c \cdot g(n)$ -t
- **Théta ($\Theta(g(n))$) - Aszimptotikus Éles korlát, közre fogva**
 - azt jelenti, hogy a függvény alulról és felülről is korlátozva van $g(n)$ konstans szorosaival, tehát az algoritmus nagyságrendileg pontosan úgy skálázódik, mint $g(n)$

Theta:

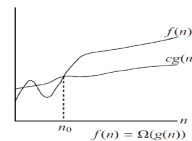
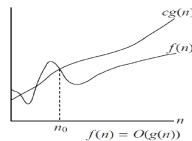
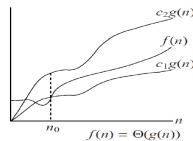
$\Theta(g(n)) = \{f(n) : \text{létezik } c_1, c_2 \text{ és } n_0 \text{ pozitív állandó úgy, hogy}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$

Ordó:

$O(g(n)) = \{f(n) : \text{létezik } c \text{ és } n_0 \text{ pozitív állandó úgy, hogy}$
 $0 \leq f(n) \leq c g(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$

Omega:

$\Omega(g(n)) = \{f(n) : \text{létezik } c \text{ és } n_0 \text{ pozitív konstans úgy, hogy}$
 $0 \leq c g(n) \leq f(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$



ELEMI ADATSZERKEZETEK

- az adatszerkezet adatok tárolására és szervezésére szolgáló módszer, mely lehetővé teszi a hatékony hozzáférést és módosítást
- elemi adatszerkezetek (nem bonthatóak tovább) közé a tömböket, láncolt listákat, valamint ezek speciális változatait, mint a verem és a sort soroljuk
- absztrakt adatszerkezet
 - egy adatszerkezet egy absztrakt adatszerkezet megvalósítása, azaz implementációja (egy absztrakt adatszerkezetnek lehet több implementációja)
 - az absztrakt adatszerkezet tartalmazza az elérhető műveleteket: tárolás, elérés, keresés, szervezés, törlés, beszúrás, min, max, megelőző és rákövetkező
 - pl: Lista, Prioritás sor

LISTA

- egy olyan absztrakt adatszerkezet ahol lineárisan követik egymást az adatok és egy érték akár többször is szerepelhet
- két fő megvalósítása: **tömbök** és **láncolt listák**

Tömb

- lista absztrakt adatszerkezet olyan megvalósítása ahol az elemeket közvetlenül lehet elérni az index-ük segítségével (közvetlen elérésű lista)
- folytonos memóriaterületen tárol azonos típusú elemeket
- fajtái:
 - statikus tömb
 - fix méret, ami deklarációkor dől el
 - dinamikus tömb
 - ha betelik akkor $2 \cdot n$ méretű tömb jön létre és a tartalma oda másolódik
 - ha pedig nincs kihasználva (pl: $n / 4$ alá esik a kihasználtság) akkor pedig $n / 2$ méretű tömb jön létre és oda másolódik a tartalma
- műveletek időigénye
 - beszúrás: $O(n)$
 - törlés: $O(n)$
 - keresés: $O(n)$ vagy $O(\log n)$
 - attól függ, hogy rendezett-e a tömb (ha rendezett akkor $O(\log n)$)
 - i-edik elem (indexelés miatti elérés): $O(1)$
- előnye a gyors elem elérés indexeléssel, hátrány a beszúrás és a törlés költségessége

Láncolt lista (linked list)

- lista absztrakt adatszerkezet megvalósítása
 - az elemei nem egy folytonos memóriaterületen tárolódnak mint a tömb esetén
 - minden eleme tartalmaz egy adatmezőt (key) és egy vagy több mutatót (pointer)
 - pl: head elem -> adat pointer -> adat pointer -> null
 - előnye, hogy nem telhet be

- fajtái
 - egyszeresen láncolt lista
 - minden elem tartalmazza a következő pointerét ami őt követi (utolsó elem mutatója null)
 - kétszeresen láncolt lista
 - minden elem tartalmazza az őt megelőző elem pointerét és a rákövetkező elem pointerét is
 - törlést könnyíti, hogy tudja a prev, megelőző pointert is
 - körkörös láncolt lista
 - olyan mint a kétszeres, csak az első elem megelőző pointere nem null lesz, hanem az utolsóra mutató pointer, míg az utolsó elem rákövetkező pointere nem null lesz hanem az első, head elem pointere lesz
- kiegészíthető egy **őrszemmel** (sentinel), ami egy speciális csomópont ami segít elkerülni a szélsőértékek speciális kezelését, úgy hogy tartalmazza azt, hogy üres-e a lista, illetve, hogy hol van a lista vége (végemutató)
 - a lista implementációját egyszerűsíti, ha egy fiktív elemet (sentinel) helyezünk el a lista végén (vagy körkörös lista esetén a fej és a vég közé)
 - így megszűnnek a határesetek (pl. üres lista kezelése), nem kell NULL pointereket vizsgálni, ami gyorsítja a kódot
- műveleteinek időigénye
 - beszúrás
 - $O(1)$ ha elejére, vagy ha van végemutató
 - $O(n)$ ha végére és nincs végemutató
 - törlés
 - $O(1)$ ha tudjuk mi van előtte
 - $O(n)$ ha meg kell keresni, vagy id alapján töröljük
 - keresés: $O(n)$
 - i-edik elem: $O(i)$, általánosítva $O(n)$
- előnye a gyors elem beszúrás és törlése

Verem (stack)

- tömbbel (egy veremmutató (top) segítségével) vagy láncolt listával
- LIFO - Last In First Out
 - csak ahhoz lehet hozzáférni amit utoljára raktunk bele
- túlcsordulás (overflow) és alulcsordulás (underflow) figyelése szükséges
 - overflow = fix méretű verem esetén ha tele van és push művelet jön
 - underflow = üres verem esetén ha pop művelet jön
- zárójelezés helyes-e arra vermet használunk
- minden művelet $O(1)$ időigényű
 - beszúrás (push) - verem tetejére
 - törlés (pop) - verem tetejéről
 - top (törlés nélkül a tetején lévő elem megnézése)
 - isEmpty (üres-e a verem)

Sor (queue)

- **cirkuláris tömbbel** (körkörös láncolt lista) vagy **listával** implementálható
 - cirkulálás tömb esetén a tömb eleje és vége összeér, hogy ne fogyjon el a hely a törlések után
 - körkörös puffert, head és tail pointerekkel, mutatókkal
- FIFO - First In First Out
 - ami először került bele, csak ahhoz van hozzáférés
- minden művelet $O(1)$
 - beszúrás (enqueue) - sor végére
 - törlés (dequeue) - kivétel sor elejéről

PRIORITÁS SOR (PRIORITY QUEUE)

- egy absztrakt adatszerkezet
- olyan mint a sor, csak a prioritásuk szerint adja vissza belőle az értékeket
 - prioritást meghatározó függvény megadása (default növekvő sorrend)
 - FONTOS, **nem rendezetten tárolja, csak prioritás szerint adja vissza**
- műveletek időigénye:
 - beszúrás: $O(\log n)$
 - törlés: $O(\log n)$
 - max (vagy min): $O(1)$

Kupac (heap)

- ez egy hatékony megvalósítása a prioritás sornak
- egy **majdnem teljes bináris fa**, ahol fenn áll a **kupac tulajdonság**
 - max kupac, azaz minden csúcsa legalább akkora mint a gyerekei (csúcs $\geq$ gyerek), a gyökér csúcs a legnagyobb értékű
 - min kupac, azaz minden csúcsa kisebb vagy egyenlő mint a gyerekei (csúcs $\leq$ gyerek), a gyökér csúcs a legkisebb értékű

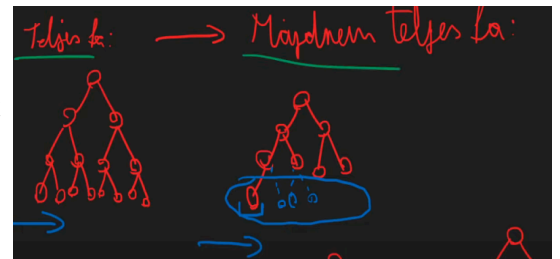
BINÁRIS KERESŐFÁK

Fa

- a fák hierarchikus adatszerkezetek (nem lineáris mint a tömb, láncolt lista, stb.)
 - egy összefüggő körmentes gráf lényegében
- gyökér (root): a fa legfelső pontja
- levél (leaf): olyan csomópont melynek nincs gyereke
- szülő és gyerek: egy csomópont feletti elem a szülő, az alatta lévők a gyerekek
- magasság (h): a gyökértől a legtávolabbi levélig vezető út hossza (élek száma)
 - műveletek sebessége a magasságtól függ
 - minnél teljesebb egy fa (minden szintje teljesen ki van töltve, vagy majdnem) a magasság annál kisebb

Bináris fa

- minden csúcsának legfeljebb 2 gyereke van (bal és jobb)
- csúcsok attribútumai: parent, left, right, key
- **teljes bináris fa**
 - minden szintje teljesen kitöltött
 - minden belső csúcsnak 2 gyereke van
 - levél csúcs: nincs gyereke (fokszám = 1)
 - belső csúcs: minden ami nem levél csúcs (fokszám > 1)
 - akkor teljes ha az adatok véletlenszerűen érkeznek vagy ha speciális (pl. AVL, Piros-Fekete) fát használunk, ami extra műveletekkel (forgatásokkal) "kényszeríti" az egyensúlyt
- **majdnem teljes bináris fa**
 - csak a legalsó szintje nem teljesen kitöltött
- **kiegyensúlyozott bináris fa**
 - minden csúcs gyerekeinek magassága legfeljebb 1-el tér el
 - ellenőrzés során megnézzük minden csúcs magasságát, levél csúcsok 0 és onnan számítva
 - ezután úgy ellenőrizzük, hogy egy csúcs elrontja-e a kiegyensúlyozottságot, hogy abszolút értékbe kivonjuk a gyerek csúcsainak magasságát és ha ez a szám nagyobb mint 1 akkor elrontja
- problémája, hogy az algoritmusába nincs olyan lépés, ami visszaállítaná a teljességet vagy az egyensúlyt



Keresőfa tulajdonság

- bináris keresőfa lényege ez
- legyen x egy csúcs a fában
 - ha y az x bal oldali részfájának egy csúcsa, akkor $\text{key}[y] \leq \text{key}[x]$
 - azaz y értéke garantáltan kisebb mint x értéke
 - ha y az x jobb oldali részfájának csúcsa, akkor $\text{key}[y] \geq \text{key}[x]$
 - azaz y értéke garantáltan nagyobb mint x értéke
- emiatt a fát inorder bejárással kiolvastva rendezett sorozatot kapunk

Keresőfa bejárások

- a bejárások határozzák meg milyen sorrendbe járjuk be a fa csúcsait
- **Inorder** (bal-gyökér-jobb) bejárás
 - rekurzió: bal részfa bejárása -> gyökér feldolgozása -> jobb részfa bejárása
 - eredmény: A fa elemeit növekvő sorrendben kapjuk meg
 - a **legfontosabb** bejárás a bináris keresőfa esetén
- **Preorder** (gyökér-bal-jobb) bejárás
 - rekurzió: gyökér -> bal részfa -> jobb részfa
 - a fa másolásánál, vagy prefix kifejezéseknél hasznos

- **Postorder** (bal-jobb-gyökér) bejárás
 - rekurzió: bal részfa -> jobb részfa -> gyökér
 - a fa törlésénél (előbb a gyerekeket töröljük, aztán a szülőt), vagy postfix kifejezéseknél

Keresőfa műveletek

- a műveletek $O(h)$ időigényűek, azaz a fa mélységétől függnek
 - legjobb eset (kiegyensúlyozott fa): $h = \log_2 n$, tehát **$O(\log n)$**
 - a gyakorlatban ezért kiegyensúlyozott fákat (pl. AVL, Piros-fekete fák) használunk, amelyek garantálják a logaritmikus magasságot
 - legrosszabb eset (elfajult fa, lánc): $h = n$, tehát $O(n)$.
- **keresés**
 - rekurzívan és iteratívan is megvalósítható
 - összehasonlítsuk az értéket a gyökérrel
 - ha kisebb az érték akkor a bal részfájába lépünk tovább
 - ha nagyobb az érték akkor a jobb részfájába lépünk tovább
 - ha egyenlő a két érték akkor megtaláltuk
 - ezt addig ismétljük még nem találjuk meg, vagy levélbe nem érünk (ha ezzel sem egyezik akkor nincs benne a bináris fába)
- **beszúrás**
 - ugyanúgy mint keresésnél haladunk és megkeressük az elem helyét és oda szúrjuk be új levélként
 - ha beszúrni kívánt elem nagyobb mint a gyökér elem akkor a jobb oldal, ha kisebb a bal oldali részfába kerül
 - ily módon megtarjuk a keresőfa tulajdonságot
- **törlés**
 - megkeressük a kívánt elemet:
 - **ha levél csúcs** akkor kivesszük (null-ra állítjuk)
 - **ha nem levél csúcs és csak 1 gyereke van** akkor a törölni kívánt csúcs szülőjét simán hozzákötjük a gyerekhez, őt magát meg töröljük
 - **ha nem levél csúcs de 2 gyereke van** akkor a törölni kívánt csúcsot rákövetkezőjének vagy megelőzőjének másolatával cseréljük, ő magát pedig töröljük
 - eredeti rákövetkező vagy megelőző is törlésre kerül az előbb említett két pont szerint
 - azért csak első két pont mivel fix 1 gyereke lehet max a rákövetkezőnek és a megelőzőnek is
 - rákövetkezőnek egy jobb oldali gyereke, a megelőzőnek egy bal oldali gyereke lehet csak, mivel ha 2 lenne akkor fix nem ő lenne a rákövetkező vagy megelőző
- **max**
 - mindig a jobb oldali gyereket választjuk még nem érünk a levél csúcsig

- **min**
 - mindig a bal oldali gyereket választjuk még nem érünk a levél csúcsig
- **nagyságszerinti rákövetkező**
 - megkeressük az adott elemet aminek a rákövetkezőjét keressük és a jobbra lépünk, ha van jobb oldali gyermeke
 - ha nincs akkor felugrálunk az ősein addig, míg valamelyik nem lesz nagyobb mint ő
 - ha nincs ilyen akkor nincs rákövetkezője
 - az első jobbra lépés után folyamatosan a bal oldali gyereket választjuk
 - jobb oldali részfa legkisebb eleme a rákövetkező
- **nagyságszerinti megelőző**
 - megkeressük az adott elemet aminek a megelőzőjét keressük és balra lépünk, ha van bal oldali gyermeke
 - ha nincs akkor felugrálunk az ősein addig, míg valamelyik nem lesz kisebb mint ő
 - ha nincs ilyen akkor nincs megelőzője
 - az első balra lépés után folyamatosan a jobb oldali gyereket választjuk
 - jobb oldali részfa legnagyobb eleme a megelőző

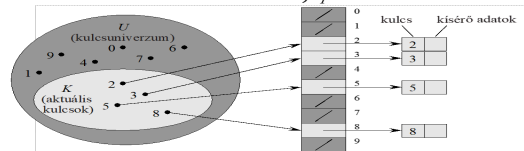
HASÍTÓ TÁBLÁK (HASH TABLE)

- a hasító táblázat a halmazok és szótár (dictionary) absztrakt adattípus hatékony megvalósítása, amely az Insert, Delete és Search műveleteket átlagos esetben $O(1)$ idő alatt végzi el
 - egy kulcshoz (érték) egy indexet rendelünk (a tömbnél pont fordítva (indexhez érték))
 - minden kulcs legfeljebb egyszer szerepel (kulcsok halmaza), de egy érték tetszőleges számban előfordulhat
 - asszociatív tömb: egészek helyett bármilyen típussal indexelhetünk
 - kulcs $\rightarrow$ érték leképzés a szótár
 - szótárak használata: DNS szerver, helyesírás ellenőrző-javító
- műveletek időigénye
 - általánosan $O(1)$ mindenre, a beszúrás mindig $O(1)$
 - törlés és keresés legrosszabb esetben $O(n)$
 - a legrosszabb eset ha az ütközéseket láncolt listákkal helyettesítjük és szinte minden kulcsot 1 helyre sorolunk be, de gyakorlatban ésszerű feltételek mellett nem jön elő és $O(1)$ -esnek mondható
- akkor használjuk ha gyors beszúrás, törlés és keresés műveletekre van szükség és rendezésre nem, illetve pl Kriptográfiában pl jelszó tárolásra, egyirányú hasító függvényen
- pl: adatbázis indexelés, caching, symbol táblák (fordítókörnyezet)
- kulcs univerzum (halmaz): U
- hasító tábla: T
 - M méretű tömb, 0-tól indexelt
 - minden kulcsot megfeleltetünk a T egyik indexével

- hasító függvény: $h = U \rightarrow \{0, 1, 2, \dots, n-1\}$
 - $h(x) = x$ kulcs hasított értéke
 - feladata leképezni egy U -beli kulcsot a T hasítótábla egy részére
 - azért van rá szükség, mivel egy nagy kulcshalmaz esetén az összes benne lévő szám szerepelhet benne, de valójában úgy sem fog minden csak egy része
 - ezért a tömb túl nagy helyet foglalna, így viszont csak U hosszával jön létre a T tömb

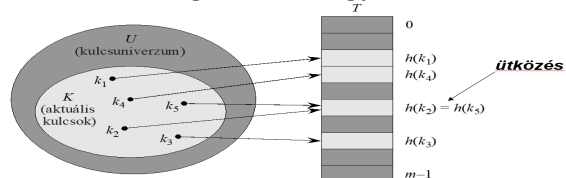
Közvetlen címzés

- ha a kulcsok egy viszonylag kicsi U univerzumból kerülnek ki, minden kulcshoz fenntartunk egy helyet egy tömbben
 - ha kicsi az U akkor éri meg csak
- előnye, hogy minden művelet $O(1)$
- hátránya, hogy ha az univerzum nagy (pl. 10 jegyű számok), de csak kevés elemet tárolunk, a memóriaigény pazarlóan nagy lesz



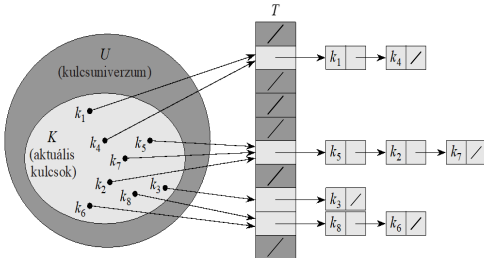
Hasítás

- hash tábla hasító függvénnyel megvalósítva
- a hasító függvény feladata leképezni egy U -beli kulcsot a T hasítótábla egy részére
 - azért van rá szükség, mivel egy nagy kulcshalmaz esetén az összes benne lévő szám szerepelhet benne, de valójában úgy sem fog minden csak egy része
 - ezért a tömb túl nagy helyet foglalna, így csak viszont U hosszával jön létre a T tömb
 - célja a minnél nagyobb véletlenszerűség és, hogy az ütközéseket minimalizálja
- $h = U \rightarrow \{0, 1, 2, \dots, m-1\}$
 - $h(x) = x$ kulcs hasított értéke
 - m a hasítótábla mérete
 - egy jó hasítófüggvény kielégíti az **egyszerű egyenletes hasítási feltételét**
 - minden elem egyforma valószínűséggel képződik le bármely részre, függetlenül attól, hogy a többiek hova kerültek
- hasító függvények típusai
 - osztásos: $h(k) = k \bmod m$
 - best practice: ha az m -et kettő hatványoktól távoli prímszámnak választjuk
 - ha $m = 12$ és a kulcs $k = 100$, akkor $h(100) = 100 \bmod 12 = 4$, ezért a 4-es indexre kerül az elem
 - szorzásos: $h(k) = m * (k * A \bmod 1)$
 - A egy konstans a $(0, 1)$ intervallumból
 - m értéke nem kritikus (ez előny, mivel nem kell feltétlen jót választani)
 - tört szám lesz általában ilyenkor az alsó egész részét vegyük
- ha T mérete kicsi de $h(x)$ nagy akkor a **túindexelés problémája** lép fel, feleslegesen sok memóriát pazarolna
 - megoldás rá az osztásos és a szorzásos hasító függvény



- a másik problémája mikor **két index a hasítótáblában ütközik**
 - $h(k_1) = h(k_2)$
 - megoldás rá a **láncolás** és a **nyílt címzés** (open addressing)

■ láncolás



- egy rés nem egy kulcsot hanem az oda hasított kulcsokból álló láncolt listát tárol
- kitöltési tényező (load factor): $\alpha = n / m$
 - n: kulcsok, elemek száma
 - m: hasítótábla mérete
 - ha nagy akkor növeljük a tömb méretét és hasítsuk újra az elemeket
 - 0.7 már nagy
 - azért kell, hogy ne legyen túl hosszú a láncolás

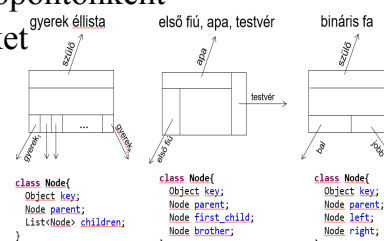
■ nyílt címzés

- minden elem magában a táblában van (nincs lista)
- ha ütközés van, új indexet próbálunk (probing)
 - lineáris próba: $h(k, i) = (h'(k) + i) \bmod m$
 - gyakori probléma az elsődleges csomósodás, klaszterezés (hosszú foglalt blokkok alakulnak ki)
 - négyzetes próba: $h(k, i) = (h'(k) + c_1 * i + c_2 * i^2) \bmod m$
 - dupla hasítás: $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$
 - ez adja a legjobb eloszlást, 2 különböző hasítófüggvényt alkalmaz

GRÁFOK ÉS FÁK SZÁMÍTÓGÉPES REPRESENTÁCIÓJA

Fák számítógépes reprezentációja

- a fák reprezentációjánál a csúcsokat és az éleket kell reprezentálnunk (lényegében maga a fa egy mutató a gyökérre), és erre több lehetőségünk is van:
- **Szülő-mutatós**
 - minden csomóponthoz csak a szülője indexét tároljuk egy tömbben
 - gyorsan megmondható, ki a szülő, és az elemek "felfelé" könnyen bejárhatók
- **Gyerek éllista:** tartalmazza magát a kulcsot, a szülőt, és a leszármazottait listában
 - minden csomóponthoz tartozik egy lista, amely a közvetlen gyerekeit tartalmazza
- **Első fiú, apa, testvér:** tartalmazza a kulcsot, a szülőt, egy gyereket, és egy testvérét
 - másnéven bal gyerek - jobb testvér
 - ez a leghelytakarékosabb módja az általános fák tárolásának
 - minden csomópontnak két mutatója van, az egyik a legelső gyerekére mutat, a másik pedig a tőle jobbra álló testvérére
 - bármilyen fokszerű fa ábrázolható fixen 2 mutatóval csomópontonként
- **Bináris fa:** tartalmazza a szülőt, a kulcsot, a jobb és a bal gyerekeket



Gráfok számítógépes reprezentációja

- **Egyszerű lista vagy tömb** (csúcs + élek halmaza)

- az összes (u, v) élt tartalmazza
- időigénye: $O(E)$
 - létezik-e (u, v) él, összes él listázása, egy csúcs szomszédainak listája
- tárigény: $\Theta(E)$
- olyan algoritmusoknál használjuk, ahol az éleket sorba kell rendezni

Csúcsok tömbje: $[A, B, C, D]$

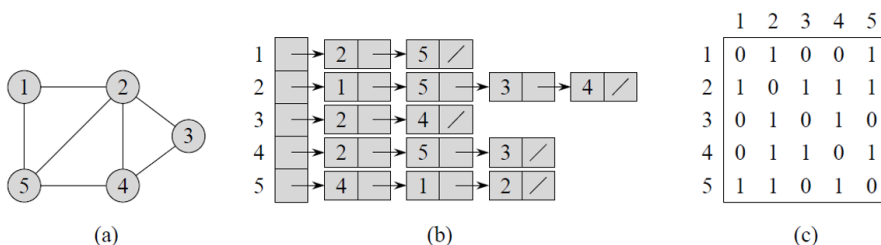
Élek listája (párok): $[(A, B), (B, C), (C, D), (D, A)]$

- **Szomszédsági mátrix**

- egy $V * V$ méretű mátrix, ahol az $A[i,j]$ elem értéke **1 (vagy a súly)**, ha van él egyébként 0 (vagy ha súlyozott akkor olyan kell amit nem vehet fel súlyként, pl null vagy ha csak pozitív súly lehet akkor akár -1)
- pl $A \rightarrow B$, akkor $M[A][B] = 1$
- ha irányítatlan a gráf akkor a szomszédsági mátrix főátlóra szimmetrikus lesz
- előny: Gyors ($O(1)$) eldönteni, hogy két csúcs között van-e él
 - akkor jó választás sűrű gráfoknál, ahol E közel van V^2 -hez, vagy ha nagyon sokszor kell él létezésre rákérdezni
- hátrány: Nagy a memóriagigénye ($O(V^2)$), akkor is ha kevés az él

- **Szomszédsági lista**

- ha két csúcs között van él, akkor azokat felvesszük (irányítatlan)
- ha az adott csúcsba vezet él, akkor azt felvesszük (irányított)
- minden csúcshoz tartozik egy lista a szomszédairól
 - tömbbe annyi férőhely ahány csúcs van és minden retesz tárol egy láncolt listát
- előny: Memóriahatékony ($O(V+E)$)
 - ritka gráfokhoz (kevés él, kb az élek száma annyi mint a csúcsok száma, $E \ll V^2$) ez a legjobb
 - gyakorlatban általában ez igaz ezért is gyakorlatban inkább ezt használjuk
- hátránya: Annak eldöntése, hogy létezik-e egy konkrét él két csúcs között lassabb $O(V)$, azaz $O(n)$



22.1. ábra. Irányítatlan gráf kétféle ábrázolása. (a) Egy G irányítatlan gráf 5 csúccsal és 7 éllel. (b) G ábrázolása szomszédsági listákkal. (c) G ábrázolása csúcsmátrixszal.

	Létezik (u,v) él?	összes él listázása	egy csúcs szomszédainak listája
csúcsok+élek halmaza	$\Theta(E)$	$\Theta(E)$	$\Theta(E)$
szomszédsági mátrix	$\Theta(1)$	$\Theta(V ^2)$	$\Theta(V)$
szomszédsági lista	$\Theta(\text{fokszám})$	$\Theta(E)$	$\Theta(\text{fokszám})$